

Natural Language Processing (NLP)

Study Notes for Testing an Intelligent Study Assistant

These notes cover basic NLP concepts, text preprocessing, embeddings, semantic search, and Retrieval-Augmented Generation (RAG). The document is intentionally text-based so it can be used to test PDF text extraction in a student project.

Learning objectives

- Explain what NLP is and identify common applications.
- Describe tokenization, stop-word removal, stemming, and lemmatization.
- Understand the difference between keyword search and semantic search.
- Explain how embeddings, vector databases, retrieval, and RAG work together.

1. Introduction to NLP

Natural Language Processing (NLP) is a field of Artificial Intelligence that enables computers to process, analyze, and work with human language. Human language can be written or spoken. NLP combines ideas from computer science, linguistics, statistics, and machine learning.

Common applications

- **Sentiment analysis:** identifies opinions or emotions in text, such as positive, negative, or neutral reviews.
- **Machine translation:** translates text from one language to another.
- **Chatbots:** respond to questions using rules, retrieval, or language models.
- **Text summarization:** produces a shorter version of a longer document while preserving key information.
- **Information extraction:** identifies useful details such as names, dates, locations, and organizations.

Typical NLP pipeline

A typical NLP pipeline starts with collecting text data. The text is then cleaned and preprocessed. Next, it is converted into a representation that a computer model can use. An algorithm performs a task such as classification, retrieval, or generation. Finally, the output is evaluated.

Example: In a movie-review classifier, the input is a review, preprocessing prepares the text, a model predicts a sentiment label, and evaluation measures how often the prediction is correct.

2. Text Preprocessing

Text preprocessing prepares raw text for analysis. The required steps depend on the task. For example, removing stop words may help a keyword-based task, but removing every stop word can damage meaning in some questions or sentences.

Important techniques

Tokenization: splits text into smaller units called tokens. Tokens may be words, subwords, or punctuation marks. For example, “NLP is useful!” may be tokenized into “NLP”, “is”, “useful”, and “!”.

Lowercasing: converts uppercase letters to lowercase. This can make “Model” and “model” match, although capitalization can sometimes carry useful information.

Stop-word removal: removes selected frequent words such as “the”, “is”, and “and”. A stop-word list is language- and task-dependent.

Stemming: reduces words to a rough root form by removing endings. For example, “connected”, “connecting”, and “connection” may be reduced to forms related to “connect”. The result may not be a valid word.

Lemmatization: maps a word to its dictionary base form using linguistic information. For example, “running” may become “run” and “better” may become “good”, depending on context and the tool.

Why preprocessing matters

Preprocessing can reduce noise and make text easier to analyze. However, it should not be applied blindly. In a question-answering system, words such as “not”, “without”, and “except” can change the meaning of a query and should usually be preserved.

3. Text Representations and Embeddings

Machine-learning systems need numerical representations of text. A simple method is Bag of Words (BoW), which represents a document using word counts. TF-IDF gives greater weight to words that are important in a document but less common across the whole collection.

Word or sentence embeddings are dense numerical vectors designed to capture aspects of meaning. Embedding models can place sentences with similar meanings near each other in vector space, even when they do not use exactly the same words.

Example

Query A: "How can I reset my password?"

Query B: "Steps to recover account access."

These sentences use different words but may have similar intent. An embedding model can help a search system recognize that relationship.

Cosine similarity

Cosine similarity compares the angle between two vectors. A larger value generally indicates that the vectors point in a more similar direction. It is commonly used to compare text embeddings during semantic search. The score is a measure of vector similarity, not a guarantee that two passages are factually equivalent.

Keyword search vs. semantic search: keyword search looks for matching words or terms. Semantic search uses representations of meaning to retrieve passages that are relevant even when the wording differs.

4. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is an approach that combines information retrieval with text generation. Instead of relying only on information learned during model training, a RAG system retrieves relevant passages from a supplied collection and uses them as context when generating an answer.

Main stages of a RAG system

- **Document loading:** read study material from PDF or other supported formats.
- **Text extraction and cleaning:** obtain readable text and remove extraction noise where appropriate.
- **Chunking:** divide long documents into smaller passages so that relevant sections can be retrieved.
- **Embedding:** convert each passage into a numerical vector.
- **Indexing:** store vectors in a searchable index, such as a FAISS index.
- **Retrieval:** embed the user's question and find passages with similar vectors.
- **Generation:** give the question and retrieved passages to a language model to draft an answer.
- **Source display:** show the document name or page number so the user can check the answer.

Why use chunks?

A long PDF may contain many unrelated topics. Chunking allows the system to retrieve only the sections that appear relevant to a question. Very small chunks may lose context, while very large chunks may contain too much unrelated material. Chunk size and overlap should be tested for the chosen documents.

5. Evaluating a Study Assistant

Evaluation helps determine whether the system retrieves relevant information and answers questions reliably. A project should use a small set of test questions whose expected answers can be checked against the source notes.

Possible evaluation measures

Retrieval relevance: manually check whether the retrieved passages contain information needed to answer the question. Precision at k can measure the fraction of the top k retrieved passages judged relevant.

Answer correctness: compare generated answers with reference answers or a marking rubric. A human reviewer can check factual accuracy, completeness, and whether the answer is supported by the retrieved source.

Faithfulness: check whether the answer is supported by the retrieved passages rather than introducing unsupported claims. Displaying page references makes manual verification easier.

Response time: record how long the system takes to retrieve passages and produce an answer. Report the number of test questions and the test setup when presenting results.

Limitations and responsible use

A RAG system can retrieve the wrong passage, miss important context, or generate an incorrect answer. Scanned PDFs may require OCR if they contain images rather than selectable text. Embedding models and language models may need an initial download, and generation may require a separate model or service. Users should verify important answers against the original study material.

Quick revision questions

- What is the difference between stemming and lemmatization?
- How does semantic search differ from keyword search?
- Why does a RAG system split documents into chunks?
- What role does FAISS play in a vector-search pipeline?
- Why should an intelligent study assistant show source page numbers?